Gleam Unified Estimation Strategy and Cutting-Edge Machine Learning Architecture

1. Current Strategy

The transition of exterior property cleaning from a discretionary aesthetic commodity into an enterprise-grade Home Asset Management ecosystem necessitates an architecture rooted in deterministic unit economics and rigorous mathematical frameworks.[1] The current architectural strategy for Gleam is explicitly governed by a Single Source of Truth (SSOT) defined within a five-tier DevSecOps topology, which strictly segregates unverified frontend data collection from the backend proprietary pricing logic.[1] The overarching philosophy is built upon the "1% Rule," which frames environmental factors—such as high-altitude ultraviolet exposure, wildfire ash deposition, and high-parts-per-million hard water—as structural threats to a home's lifespan, thereby justifying premium subscription-based maintenance models.[1]

To operationalize this philosophy, the estimation process revolves around the construction of a "Digital Twin." This central data payload replaces human guesswork and flat-rate estimating with a dynamic JSON-based profile constructed from six distinct data layers.[1] The Persona and Conversion Twin captures digital identity and attribution parameters to compute a lead score.[1] The Actuarial and Structural Twin pulls deterministic data, such as above-grade and below-grade square footage, alongside the year built, via real estate application programming interfaces (APIs) to establish a rigid quoting baseline.[1] The Sensory and Chemistry Twin utilizes artificial intelligence and large language models to scrape unstructured visual data, capturing exact fenestration geometry, siding materials, and window coatings.[1] The Environmental and Geospatial Twin maps high-resolution topological data, including roof pitch and local water hardness, to calculate physical friction.[1] The Friction, Hazard, and Legal Twin tracks operational variables like interior access obstructions and generates automated tripwires for regulatory compliance, such as Occupational Safety and Health Administration (OSHA) fall-protection limits.[1] Finally, the Relationship and Financial Twin serves as the customer relationship management and ledger reality, tracking the exact effective hourly rate per job.[1]

The current end-to-end operational flow follows a Zero-Trust ingestion model designed to protect intellectual property and prevent malicious payload injection. At the Zero-Trust Presentation Edge, a "dumb frontend" collects primitives such as addresses and requested services, utilizing Google reCAPTCHA v3 and session-based tracking parameters without executing any pricing mathematics client-side.[1] Inputs then pass through an API Gateway and Sanitization layer, operating via a Make.com routing engine. Here, Poka-Yoke data clamps neutralize hyper-scale errors by hard-clamping anomalous user inputs, such as forcing any

manual square footage entry exceeding fifteen thousand back down to a safe threshold.[1] During the Vision Audit and Digital Twin Hydration phase, the backend queries real estate APIs for actuarial priors and passes Google Street View and satellite imagery to a Gemini Vision API for sensory auditing.[1]

Pricing reconciliation is subsequently executed using a strict cost-plus mathematical schema. Execution time and costs are calculated using defined Variable Task Modifiers (VTMs) applied to locked Base Unit Costs.[1] The system structurally isolates overhead and margin. The Base Unit Cost is derived from physical time measurements plus transition buffers, loaded with a technician labor rate of thirty-six dollars per hour, consumables, risk adders, and a flat fifteen percent indirect overhead multiplier.[1] VTMs modify these base times based on the physical realities of the work. Multiplicative modifiers apply to tasks where the overall speed of unit completion changes, such as Height Level, Access Difficulty, Roof Pitch, and Contamination Level.[1] Additive modifiers apply to discrete procedural blocks added to a service, such as Window Soiling Level, which adds explicit minutes and costs per pane.[1]

The architecture enforces strict guardrails to protect unit economics. The system mandates a minimum job floor of one hundred and twenty-five dollars, applies a strict discount cap of twenty percent across combined promotional offers, and isolates the Crew Efficiency Factor (CEF) purely to schedule duration forecasting.[1] This isolation ensures the customer-facing price remains mathematically identical regardless of whether a solo technician or a multi-person crew completes the work.[1] Escape hatches are explicitly defined to block automated binding when hazardous constraints are met. Triggers such as four-plus stories, post-construction hazmat environments, steep roof pitches exceeding a 9/12 ratio, or code-noncompliant dryer vents immediately route the user to an onsite assessment workflow.[1] Finally, job cards are generated via the ServiceM8 REST API using a rigid two-step Company and Contact creation process, generating unique SKU strings that log exact service parameters for downstream analytics.[1]

2. Gaps/Risks

A comprehensive forensic review of the provided single source of truth documentation, architectural schemas, and integration contracts reveals several critical contradictions and architectural risks. If these discrepancies are migrated into the production machine learning architecture without resolution, they pose severe threats to margin integrity, API stability, and operational usability.

Current State, Contradictions, and Failure Modes

Component	Evidence	Current Operational	Known Failure Modes &	Margin Risk if

	Source	Logic	Contradictions	Unresolved
Labor Rate & Margin Target Divisor	JCF v4 vs. Pricing Schema [1]	JCF dictates a $36.00/hr labor burden with a 0.60 margin divisor targeting a 40% gross margin.[1]	The pricing math schema JSON indicates a 0.70 divisor, targeting only a 30% margin.[1] Deprecated documents incorrectly reference $53.82/hr.[1]	**Severe.** Misaligned margin divisors threaten a massive 10% gross profit erosion globally across all automated quotes.
Setup Fee Waiver Logic	Continuity Charter vs. JCF v4 [1]	The $60 setup fee represents actual logistical pass-through costs and is mandatory.[1]	Previous iterations allowed waivers for multiple category selections or subtotals exceeding $250. The current SSOT dictates waivers *only* apply for co-located "Bright Block" neighbor bookings.[1]	**High.** Erroneously waiving the setup fee based on subtotal size consumes approximately 115 minutes of unbilled logistical labor per job.
Overhead Application on VTMs	Cost Buildup Logic [1]	The 15% indirect overhead is allocated exclusively at the base cost level.[1]	Code applying overhead dynamically to VTM cost adders double-counts administrative and IT burdens	**Moderate.** Systematically inflates customer prices on highly complex jobs, eroding close rates and

			on every incremental modifier.[1]	competitive positioning.
Crew Efficiency Factor (CEF) Leakage	Guardrails Section [1]	CEF reduces calendar-block time by roughly 40% for two-technician dispatch routing.[1]	Leakage of the CEF multiplier into the customer-facing pricing math artificially deflates the quote based on crew size.[1]	**Severe.** Passing efficiency gains directly to the customer destroys multi-technician margins and invalidates the cost-plus architecture.
ServiceM8 API Custom Field Exhaustion	ServiceM8 Integration [1]	ServiceM8 enforces a hard limit of exactly ten custom fields per object type.[1]	Pushing granular vision observations and VTM breakdowns into custom fields exhausts the budget, preventing the addition of critical routing fields like quote_mode.[1]	**Moderate.** Results in API request rejections, orphaned job records, and the loss of essential operational routing data.
ServiceM8 Badge Array Overwrites	ServiceM8 Integration [1]	Make.com assigns marketing and hazard badges to the ServiceM8 job via a PATCH request.[1]	A direct PATCH request overwrites the entire existing array, instantly deleting any badges previously applied by human operators.[1]	**Low.** Causes loss of manual operational data and creates blind spots regarding onsite hazards for dispatched technicians.

VTM Classification Errors	VTM Formulas [1]	Gutter soiling levels are multiplicative, while window soiling levels are additive.[1]	Treating window soiling multiplicatively, or gutter soiling additively, violates the physical mechanism of the work.[1]	**Moderate.** Unjustified price inflation lowers conversion rates on core services, or underpricing leads to unbilled labor on heavy debris jobs.

SSOT Change Requests and Resolutions

To ensure absolute alignment between the mathematical schemas and the business objectives, explicit resolutions must be enforced within the data contracts.

SSOT Change Request 1: Standardization of the Margin Divisor. The target_margin_divider within the gleam_pricing_math_schema.json must be formally updated to 0.60. Maintaining the current 0.70 value specified in the schema directly conflicts with the executive mandate outlined in the Blueprint to achieve a true forty percent gross margin on aggregate costs.[1] Resolving this conflict protects the core margin targets of the enterprise. The margin impact of this change is the immediate preservation of a ten percent gross profit across the entire automated quoting pipeline.

SSOT Change Request 2: Custom Field Budget Migration. Given that ServiceM8 strictly limits custom fields to ten per object, low-value analytical fields must be decommissioned from the custom field architecture.[1] Fields such as customfield_execution_time_roofing_material and detailed VTM array strings must be formatted as concatenated text and injected directly into the standard job_description free-text field.[1] This migration preserves the strict API budget for high-value routing parameters required by the new machine learning architecture, such as quote_mode and glass_type_confirmed. This change prevents API rejection failure modes and carries a neutral margin impact while drastically improving system stability.

SSOT Change Request 3: Enforcement of the Service-Group Dispatcher Pattern. The codebase must immediately deprecate any single, universal adjustment function. The architecture must implement a Service-Group Dispatcher that applies variables based on distinct physical mechanisms.[1] If a modifier changes the speed at which a technician completes every subsequent unit—such as operating on a steep roof pitch or navigating a highly obstructed yard—the modifier must be programmed as multiplicative.[1] If a modifier adds a discrete block of work independent of other conditions, such as scraping heavy tree sap off a window pane, it must be programmed as an additive block.[1] Enforcing this architectural pattern

prevents the systematic over-quoting that occurs when additive factors are erroneously multiplied against baseline durations.

3. Best-Practice Research

To modernize the digital twin estimation engine and transition from heuristic rules to a truly cutting-edge, data-driven architecture, external literature points toward the integration of hybrid estimation systems, uncertainty quantification, advanced multi-view computer vision fusion, and robust machine learning operations.

Hybrid Estimation Systems and Bayesian Reconciliation

Modern automated valuation and property estimation models treat ground truth not as a singular known variable, but as a distribution of probabilities.[2] Hybrid estimation systems construct accurate predictions by blending actuarial priors—such as public tax records or historical municipal data—with sensory likelihoods generated by perception algorithms via Bayesian updating.[3] In physical home services, reliance on singular data streams is inherently flawed; property databases frequently suffer from staleness due to unregistered home additions, while vision models can fail under varied lighting conditions.[5] Bayesian frameworks elegantly resolve this by treating the artificial intelligence vision output as an observed event, weighting both the actuarial prior and the visual likelihood by their historical epistemic uncertainty.[5] This mathematical reconciliation ensures that the posterior estimate naturally gravitates toward whichever data source exhibits the highest statistical confidence in a given geographic cluster.

Uncertainty Quantification via Conformal Prediction

Traditional deep learning regression models yield point estimates, which are highly dangerous in margin-sensitive applications because they lack a quantified measure of confidence.[7] If a model predicts twenty-four window panes with absolute certainty, but is actually failing to account for heavy occlusion, the resulting binding quote destroys profitability. Conformal prediction provides a distribution-free statistical framework that wraps around any black-box machine learning model to output statistically guaranteed prediction intervals.[8] For example, instead of predicting exactly twenty-four panes, the system provides a ninety percent mathematical guarantee that the true pane count lies between twenty-two and twenty-six.[10] In housing markets characterized by extreme spatial heteroscedasticity, split conformal prediction methods dynamically adjust interval widths based on local data variance and specific feature attributes.[2] If an algorithmic vision model encounters a property partially occluded by dense vegetation, conformal prediction algorithms automatically widen the prediction interval, allowing automated routing systems to elegantly transition the user to a range quote rather than risking a firm financial bind.[9]

Multi-View Vision Fusion and Occlusion Handling

Estimating precise fenestration geometry and roof topologies from single, ground-level images is notoriously error-prone due to vegetation occlusion, vehicular obstruction, and extreme perspective distortion.[13] The state-of-the-art approach in architectural computer vision is multi-view fusion.[15] This methodology involves generating a top-down structural footprint from satellite imagery and projecting street-view feature extractions onto the identified structural planes.[17] By fusing high-dimensional features from disparate angles, multi-view approaches effectively peer through occlusions.[19] Furthermore, modern systems leverage density map regression techniques for counting tasks in densely packed or occluded scenes, systematically outperforming traditional bounding-box object detectors by predicting a continuous density map where pixel intensity corresponds to target concentration.[13] The integration of architectural symmetry constraints—assuming repeated window patterns across identical facades—allows algorithms to infer missing elevations and resolve occluded areas with high structural plausibility.[20]

Asymmetric Loss Functions

In pricing estimation and physical labor dispatching, the cost of predictive error is rarely symmetric.[21] Underestimating labor time by thirty minutes directly destroys profit margins and disrupts subsequent scheduling blocks, whereas overestimating by thirty minutes slightly reduces top-of-funnel conversion probabilities but maintains absolute financial safety and operational buffer.[22] Statistical best practices dictate the use of asymmetric loss functions, such as the Linear Exponential (LINEX) loss or quantile regression pinball loss, during model training.[23] These functions inherently penalize underestimations significantly more heavily than overestimations.[21] By training the machine learning architecture on an asymmetric loss surface, the predictive distribution is conservatively shifted to protect unit economics, ensuring the automated system naturally rounds up when faced with deep uncertainty.[26]

Feature Stores and Schema-Driven MLOps

Scaling a predictive pricing architecture requires a fundamental shift from monolithic scripts to a schema-driven Machine Learning Operations (MLOps) environment. Utilizing a centralized feature store ensures point-in-time correctness, allowing batch features—such as historical real estate data—to seamlessly merge with real-time streaming features—such as user-submitted observations—without introducing data leakage or training-serving skew.[28] Furthermore, continuous drift monitoring is essential for maintaining pricing integrity. Implementing metrics such as the Wasserstein distance or population stability indices protects the system against environmental concept drift, such as changes in local foliage occlusion over seasons, and alerts engineering teams to model degradation before margins are impacted.[31]

Human-in-the-Loop (HITL) Design

Rather than forcing an algorithmic model to guess on highly uncertain data, modern robust architectures rely heavily on Human-in-the-Loop workflows.[33] When the conformal prediction interval width exceeds a predefined risk threshold, or when classification confidence scores for hazardous conditions fall below acceptable limits, the workflow suspends algorithmic binding.[35] The imagery and the uncertain prediction are subsequently routed to a human auditor.[36] The auditor's correction is then fed back into the model pipeline via active learning mechanisms, continuously refining the predictive boundary, updating the feature store, and improving future automated decisions without sacrificing customer trust or financial safety.[33]

Synthesis: What to Borrow and What to Avoid

The unified strategy must borrow the integration of multi-view fusion for architectural analysis, leveraging both satellite footprints and street-level imagery to combat occlusion. The architecture must adopt split conformal prediction to generate mathematically sound prediction intervals, driving the routing logic for quote modes. Furthermore, the models must be trained using asymmetric quantile loss functions to inherently protect against the catastrophic margin erosion associated with undercounting physical units. Conversely, the strategy must explicitly avoid relying on single-view bounding box detection for counting fenestration, as it degrades rapidly under suburban vegetation. The architecture must also avoid symmetric mean squared error training objectives, which fail to account for the asymmetric financial risks of service dispatching.

4. Proposed Unified Strategy

To satisfy the single source of truth constraints while achieving state-of-the-art accuracy, the proposed strategy adopts a highly refined learning-to-estimate quantities architecture. This strategy utilizes constraint layers, conformal prediction uncertainty modeling, and rigid business rule guardrails to replace static heuristics with a calibrated stochastic model bounded by the enterprise's deterministic financial rules.

4.1. System Architecture and Data Contracts

The architecture functions as a unidirectional pipeline, strictly enforcing the five-tier DevSecOps philosophy and separation of concerns.[1]

The ingestion layer begins with inputs drawn from public property data APIs, providing actuarial square footage and year-built priors. Spatial clusters, defined by ZIP code embeddings, provide historical property type priors. Simultaneously, the Zero-Trust presentation edge collects user answers, requested service selections, and raw imagery metadata.[1] All inputs are routed into the Machine Learning Feature Store, which normalizes the data against the gleam_sku_registry_schema.json format.[1]

The core predictive engine processes these unified features through a dual-stream architecture. Stream A outputs continuous predicted quantities, explicitly preserving the

architectural primitives by outputting panes, architectural openings, linear footage, and square footage as entirely distinct numerical targets. Stream B executes multi-class classification to predict the probability distribution of Variable Task Modifiers, including Height Levels, Soil Levels, Access Difficulty, Roof Pitch, and Contamination Levels.[1]

These outputs pass through an uncertainty quantification layer, yielding mathematically guaranteed confidence scores and conformal prediction intervals. The pricing engine subsequently maps these intervals against the deterministic gleam_pricing_math_schema.json, applying the appropriate modifiers to base unit costs and calculating the discrete setup fee and discount logic.[1] Finally, the system outputs a JSON payload containing the finalized quote mode—bind, range, or onsite—along with generated SKU line items, total pricing, and isolated scheduling durations.[1]

4.2. Modeling Approach

When designing the core algorithmic engine, two primary architectures were evaluated. Design 1 relies on strict heuristic rules blended with calibrated regression priors, basic vision counting, and standard Bayesian reconciliation. While computationally inexpensive, Design 1 fails to adequately handle complex architectural occlusion and relies too heavily on static ZIP-code multipliers, which frequently lead to spatial overfitting and data leakage.

Therefore, the recommended architecture is **Design 2: Learning-to-Estimate Quantities with Constraint Layers, Conformal Prediction, and Rule Guardrails**.

In this design, the targets predicted are absolute physical quantities (e.g., specific pane counts and linear footage of gutters) and categorical risk tiers (e.g., the probability of the property exceeding three stories). Uncertainty is represented utilizing split conformal prediction, generating prediction intervals that bound the true value with a user-specified probability (e.g., a ninety percent coverage guarantee).[10]

To handle partial visibility and occlusion, the model utilizes multi-view point cloud alignment and density map regression.[13] By fusing the overhead satellite footprint with multi-angle street view data, the algorithm infers missing elevations using learned architectural symmetry constraints.[20] If an elevation is entirely obscured by mature trees and symmetry cannot be confidently applied, the model's epistemic uncertainty spikes, causing the conformal prediction interval to drastically widen.[12] Furthermore, image staleness is addressed by applying an exponential decay weight to older photographs within the feature store, ensuring that recent user-uploaded imagery or recent municipal surveys exert stronger influence on the posterior distribution. To avoid spatial leakage and overfitting by ZIP code, spatial features are embedded via target encoding with high regularization parameters, preventing the model from merely memorizing affluent neighborhoods and forcing it to rely on actual structural features.

4.3. Asymmetric Reconciliation Logic

When actuarial tax data and computer vision outputs conflict, the system does not resort to simple mathematical averaging. Instead, the architecture utilizes a Bayesian updating framework tilted by an asymmetric loss function, specifically quantile regression loss.[23]

The integration of asymmetric weighting is paramount to prevent undercounting. If the actuarial prior suggests a property has twenty windows, but the vision model detects twenty-two with moderate confidence, a symmetric model might predict twenty-one. However, an asymmetric model heavily penalizes underestimation, naturally pushing the posterior prediction toward the higher count to protect margin.[21] This blending yields a posterior quantity, accompanied by a precise confidence interval and structured reason codes (e.g., OCCLUSION_DETECTED_ELEVATION_C) that inform the downstream routing logic. This asymmetric reconciliation preserves a minimum of five percent gross margin that is typically lost to hidden or occluded units in flat-rate quoting environments.

4.4. Quote Mode Policy

The Quote Mode is dictated entirely by the width of the generated conformal prediction interval and the probabilistic presence of hazardous escape hatches. The policy enforces three distinct routing paths:

1. **BIND:** The Bind mode is activated strictly if the ninety percent conformal interval for the total job cost spans a financial variance of less than plus or minus twelve percent, and there is less than a one percent probability of any escape hatch (e.g., HL.4, SL.4, PT.4, NL.4) being present.[1] In this scenario, the customer is presented with a firm, binding price, and the checkout flow proceeds.
2. **RANGE:** The Range mode is activated if the conformal interval spans a variance between twelve percent and thirty-five percent. The user interface presents an asymmetric price range calculated directly from the percentiles of the posterior distribution, rather than arbitrary structural adders. The system subsequently prompts the user with "tightener" questions, such as requesting a photo of the obscured side elevation, to narrow the statistical band and drive the quote toward a Bind state.[1]
3. **ONSITE:** The Onsite mode is triggered immediately if the upper bound of any predicted VTM set touches a hazardous escape hatch (e.g., a high probability of a four-story facade), or if the baseline prediction interval width exceeds thirty-five percent due to extreme occlusion or data staleness.[1] Algorithmic pricing is completely suppressed, and the user is routed to book a mandatory sixty-dollar onsite assessment.[1]

4.5. Pricing Engine Alignment

The predicted quantities and VTM classifications are fed into the deterministic pricing engine strictly according to the rules defined in gleam_pricing_math_schema.json.[1] The architecture maps these variables to final pricing as follows:

First, base labor minutes are computed specifically for the service selected. The adjusted base

time is calculated by taking the raw field-observed time and adding the required tool transition buffers.[1] The base cost is then derived by multiplying this adjusted time by the technician hourly burden of thirty-six dollars, and adding direct consumables and assessed liability risk costs.[1]

Crucially, the fifteen percent indirect overhead allocation is applied mathematically at this step, yielding the locked base unit cost.[1] The Service-Group Dispatcher then determines VTM application. If the service demands multiplicative execution—such as the impact of height or access difficulty on the overall speed of the task—the adjusted time and cost are multiplied by the product of the applicable VTMs.[1] If the service utilizes additive execution—such as specific window soiling levels—the discrete minute and cost adders are appended linearly to the calculation.[1]

The system then calculates the service subtotal and rigorously enforces the financial guardrails. The mandatory sixty-dollar setup fee is added to every job; it is waived exclusively if the internal Bright Block neighbor-booking flag evaluates to true, reflecting genuine logistical pass-through savings.[1] Bundle and subscription discounts are evaluated, but combined discounts are mathematically capped at exactly twenty percent of the service subtotal, protecting bottom-line profitability.[1] Finally, if the resulting final price falls below one hundred and twenty-five dollars, the minimum job floor clamp activates, overriding the calculation to guarantee the minimum visit threshold.[1]

The Crew Efficiency Factor is explicitly partitioned out of all financial calculations. The customer price is calculated based on standard single-technician baselines, while the CEF modifier is routed exclusively to the scheduling payload to reduce calendar blockage duration.[1] The final output generates SKU strings perfectly aligned with the gleam_sku_registry_schema.json format, seamlessly tracking variables for downstream analytical reporting.[1]

5. Pseudocode & Data Contracts

5.1. JSON Data Contracts

The unidirectional flow of data is strictly governed by the following schema contracts.

Input Schema (Zero-Trust Frontend to API Gateway):

JSON

```
{
 "security": {
  "recaptcha_token": "token_string_alpha_numeric"
```

```
 },
 "customer": {
   "email": "homeowner@example.com",
   "zip": "80205"
 },
 "property_core": {
   "address_string": "123 Cherry Creek Drive"
 },
 "requested_services":
}
```

Digital Twin Internal Payload (Post-Vision Audit & Reconciliation):

JSON

```
{
 "property_id": "uuid_v4",
 "actuarial_priors": {
   "agsf": 2850,
   "year_built": 2004
 },
 "vision_posterior": {
   "primitives": {
     "pane_count": {"estimate": 36, "interval_lower": 34, "interval_upper": 41},
     "openings": {"estimate": 18, "interval_lower": 18, "interval_upper": 20}
   },
   "vtm_probabilities": {
     "HL": {"HL1": 0.05, "HL2": 0.90, "HL3": 0.05, "HL4": 0.0},
     "AD": {"AD1": 0.85, "AD2": 0.15, "AD3": 0.0}
   },
   "uncertainty_flags":
 }
}
```

Pricing Engine Output (To ServiceM8 & User Interface):

JSON

```
{
 "quote_id": "Q-20260226-084",
 "quote_mode": "RANGE",
 "line_items":},
    "line_price": {"point": 148.50, "range": [140.25, 169.12]}
  }
 ],
 "pricing": {
  "subtotal": 148.50,
  "setup_fee": 60.00,
  "discount_applied": 0.00,
  "final_price_point": 208.50,
  "min_job_floor_applied": false
 },
 "scheduling": {
  "duration_mins": 155,
  "cef_active": true
 }
}
```


5.2. Mathematical Formulas and Algorithmic Pseudocode

To train the machine learning models to inherently penalize underestimation, the architecture utilizes an asymmetric pinball loss function, where the hyperparameter $\tau \in (0.5, 1)$ dictates the severity of the asymmetry.[23] Let y represent the true physical pane count, and $\hat{y}$ represent the algorithm's predicted count. By setting $\tau = 0.85$, the model is mathematically penalized over five times more heavily for undercounting a window than for overcounting one.

$$L(y, \hat{y}) = \begin{cases} \tau(y - \hat{y}) & \text{if } y \geq \hat{y} \\ (1 - \tau)(\hat{y} - y) & \text{if } y < \hat{y} \end{cases}$$

Bayesian Reconciliation and Conformal Routing Pseudocode:

Python

```
def reconcile_priors_and_vision(actuarial_data, vision_features, tau=0.85):
  """
```

```
    Fuses actuarial data and vision features using a conformalized ML pipeline
    optimized under an asymmetric loss function to protect unit margins.
    """
    # X encapsulates both modalities via the central feature store
    X = FeatureStore.fetch_fused_vector(actuarial_data, vision_features)

    # Model trained with asymmetric pinball loss (tau=0.85) to prevent undercounting
    base_prediction = AsymmetricModel.predict(X)

    # Split conformal prediction yields intervals with 90% marginal coverage guarantees
    interval_lower, interval_upper = ConformalPredictor.get_interval(X, alpha=0.10)

    # Uncertainty-based routing logic calculates the percentage width of the interval
    interval_width_pct = (interval_upper - interval_lower) / base_prediction

    # Check for hazardous escape hatch probabilities
    escape_hatch_risk = VTMClassifier.predict_probability(X, target="HL4")

    if escape_hatch_risk > 0.01 or interval_width_pct > 0.35:
        mode = "ONSITE"
    elif interval_width_pct <= 0.12:
        mode = "BIND"
    else:
        mode = "RANGE"

    return base_prediction, interval_lower, interval_upper, mode
```

Service-Group Dispatcher and Pricing Enforcement Pseudocode:

Python

```
def calculate_line_item(service_id, base_time, base_cost, vtm_set):
    """
    Executes pricing math strictly aligned to gleam_pricing_math_schema.json,
    enforcing overhead isolation and margin targeting constraints.
    """
    overhead_multiplier = 1.15
    margin_divisor = 0.60 # Standardized to 40% margin per SSOT Change Request

    # Overhead is applied mathematically ONCE at the base level, never on VTMs
```

```
    loaded_base_cost = base_cost * overhead_multiplier

    if is_multiplicative_service(service_id):
        # Physical mechanism: Modifiers alter the overall speed of the task
        vtm_multiplier = vtm_set.HL * vtm_set.AD * vtm_set.PT * vtm_set.MA * vtm_set.CT
        adjusted_time = base_time * vtm_multiplier
        adjusted_cost = loaded_base_cost * vtm_multiplier
    else:
        # Physical mechanism: Additive blocks (e.g., specific window soiling levels)
        adjusted_time = (base_time * vtm_set.HL * vtm_set.AD) + vtm_set.SL_time
        adjusted_cost = (loaded_base_cost * vtm_set.HL * vtm_set.AD) + vtm_set.SL_cost

    # Final customer price targets precise gross margin percentages
    customer_price = adjusted_cost / margin_divisor
    return customer_price, adjusted_time
```

6. Evaluation & Rollout Plan

To safely migrate from the legacy heuristics to this unified stochastic architecture without jeopardizing active cash flow or dispatch operations, a strict Machine Learning Operations evaluation protocol and a four-phase implementation roadmap are required.

Evaluation and Continuous Monitoring Metrics

The MLOps environment will track specific metrics to ensure pricing integrity and model health.

Offline Metrics:

- **Quantity MAPE and Asymmetric MAE:** Mean Absolute Percentage Error measures absolute accuracy, while Asymmetric Mean Absolute Error tracks the weighted penalty for undercounting physical primitives.
- **Interval Marginal Coverage:** Statistically verifies that the ninety percent conformal prediction intervals truly capture the ground truth values ninety percent of the time on hold-out validation datasets.[10]
- **Escape Hatch Recall:** Tracks the percentage of true hazardous scenarios (e.g., steep roof pitches or four-story elevations) correctly flagged for onsite safety routing.

Online Tests:

- **A/B Routing Tests:** Segmenting top-of-funnel web traffic to measure conversion elasticity when a customer is presented with a tight range quote versus a definitive bind quote.

Continuous Monitoring:

- **Environmental Drift:** Continuous tracking of input distributions using the Wasserstein metric. An automated alert triggers retraining workflows if distribution drift—such as seasonal changes in vegetation occlusion—exceeds a 0.1 threshold.[32]
- **Vision Failure and Low-Confidence Rates:** Monitoring the frequency at which the system routes jobs to the Human-in-the-Loop or Onsite queues, ensuring the operational capacity of human auditors is not overwhelmed.

Feedback Loop Integration:

- ServiceM8 acts as the ultimate ground-truth ledger. When technicians complete a job, they adjust the final invoiced quantities. This variance between predicted arrays and actual execution is automatically piped back into the feature store to continuously recalibrate the asymmetric loss weights and conformal calibrators.

Phased Implementation Roadmap

Phase 1: ServiceM8 Foundation and API Integrity

- **Scope:** Build the rigid, deterministic Make.com to ServiceM8 pipeline.
- **Action:** Implement the correct two-call procedure (POST /company.json followed by POST /companycontact.json) to preserve the relational hierarchy.[1] Implement the Badge array GET-then-PATCH logic loop to append tags securely, preventing destructive marketing overwrites.[1] Deactivate extraneous custom fields to respect the ten-field API limit.
- **Dependencies:** Complete synchronization of the gleam_sku_registry_schema.json within ServiceM8 Materials parameters.
- **Risks & Margin Impact:** Development errors could temporarily orphan job records. Successfully implemented, this phase protects against operational data loss, carrying a neutral but protective margin impact.

Phase 2: Pricing Engine and Service-Group Dispatcher

- **Scope:** Deploy the computational logic layer enforcing gleam_pricing_math_schema.json.
- **Action:** Build the Service-Group Dispatcher, ensuring absolute mathematical separation between multiplicative and additive VTM applications. Enforce the 1.15 overhead isolation. Implement the twenty percent discount cap and the one hundred and twenty-five dollar minimum floor logic.
- **Dependencies:** Stable API integrity established in Phase 1.
- **Risks & Margin Impact:** Logic mapping errors could misprice distinct services. Successful implementation captures an immediate ten percent margin uplift by correcting the prior architecture's mathematical drift, specifically regarding the margin divisor and overhead misapplication.

Phase 3: Zero-Trust Presentation Edge (Dumb Quote Tool)

- **Scope:** Deploy the customer-facing frontend interface on Squarespace.
- **Action:** Integrate Google reCAPTCHA v3 bot mitigation. Construct the session-storage

Magic Link handoff mechanism. Ensure that absolutely no pricing logic executes within client-side JavaScript arrays.[1]

- **Dependencies:** A fully functional and load-tested Layer 2 API Gateway (The Bouncer).
- **Risks & Margin Impact:** Poor user interface design could increase checkout friction. Operationally, it protects proprietary intellectual property from competitive scraping and eliminates client-side payload tampering risks.

Phase 4: Digital Twin and Vision Audit Engine

- **Scope:** Deploy the hybrid machine learning estimation and conformal prediction engine.
- **Action:** Integrate RentCast actuarial data streams, Gemini sensory arrays, and the Bayesian reconciliation engine. Activate the Bind, Range, and Onsite dynamic routing logic based entirely on prediction interval widths. Implement the Human-in-the-Loop fallback queue for low-confidence auditing.
- **Dependencies:** The accumulation of sufficient ground-truth data from Phase 2 and Phase 3 manual technician corrections to effectively calibrate the conformal predictions.
- **Risks & Margin Impact:** The highest technical risk phase; uncalibrated intervals could route too many jobs to the range queue, lowering instant conversions. When optimized, it scales revenue dramatically by providing instant binding on high-confidence jobs while routing complex, high-variance jobs to safety gates.

7. Margin Impact Summary

Every strategic recommendation within this unified architecture is meticulously optimized to defend the unit economics defined by the single source of truth constraints. The transition from static rules to conformalized machine learning represents a fundamental fortification of the enterprise's financial integrity.

Proposed Recommendation	System Benefit	Implementation Risk	Quantified Margin Impact
Apply Asymmetric Loss Function ($T =$)	Penalizes the algorithmic model severely for underestimating quantities, effectively mitigating revenue loss from physical occlusion.[23]	May yield overly conservative estimates, slightly dampening top-of-funnel conversion rates on complex properties.	**Protects 5% to 8% Gross Margin** previously lost to unbilled logistical labor on hidden or occluded physical units.

Conformal Prediction Interval Routing	Replaces brittle point estimates with statistically guaranteed ranges, dynamically routing risky jobs to onsite assessments.[10]	Requires complex computational overhead and careful statistical calibration of the alpha parameter to balance range widths.	**Prevents 100% of catastrophic misquotes** (e.g., binding a $200 price on a $900 scope) by mathematically blocking low-confidence bindings.
Service-Group Dispatcher Architecture	Eradicates the mathematical compounding error of treating flat-adders (such as window soil levels) as speed multipliers across the base time.	Developer error in the Make.com router logic could incorrectly map service IDs to the wrong calculation array.	**Prevents systematic 15% price over-inflation** on standard exterior window jobs, protecting essential close rates and market competitiveness.
Enforce Overhead Isolation & 0.60 Margin Divisor	Overrides legacy $53.82/hr logic and applies the 1.15 overhead strictly to base costs, yielding a true 40% gross margin baseline.	Exposes conflicts in older documentation, requiring immediate and strict operational alignment across all teams.	**Instantly captures +10% Gross Profit** on all generated SKUs relative to the undetected 30% drift schema.
Human-in-the-Loop Fallback Integration	Stops the artificial intelligence from automatically binding quotes on architectural edge-cases and leverages technician corrections for active learning.[35]	Adds necessary latency to the quoting process for the specific subset of users who trigger the statistical fallback threshold.	**Eliminates margin decay from model drift** by creating an automated, self-correcting feedback loop that continuously sharpens the feature store.

Isolate Crew Efficiency Factor (CEF) to Scheduling	Ensures two-technician efficiency mathematically reduces calendar blockage duration without reducing the customer-facing invoice total.[1]	Misconfiguration in the ServiceM8 API payload could accidentally expose shorter duration times to final customer billing integrations.	**Preserves ~40% margin velocity** on multi-technician dispatch days, ensuring the enterprise retains the financial benefits of optimized routing.

Works cited

1. continuity-charter.md
2. Uncertainty quantification in automated valuation models with spatially weighted conformal prediction - arXiv, accessed February 26, 2026, https://arxiv.org/html/2312.06531v2
3. (PDF) Reconciling "priors" and "priors" without prejudice? - ResearchGate, accessed February 26, 2026, https://www.researchgate.net/publication/278797851_Reconciling_priors_and_priors_without_prejudice
4. Deriving Priors for Bayesian Prediction of Daily Response Propensity in Responsive Survey Design: Historical Data Analysis Versus Literature Review - PMC, accessed February 26, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10080219/
5. Bayesian binding and fusion models explain illusion and enhancement effects in audiovisual speech perception - PMC, accessed February 26, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7895372/
6. Bayesian Hybrid Automata: A Formal Model of Justified Belief in Interacting Hybrid Systems Subject to Imprecise Observation - DROPS, accessed February 26, 2026, https://drops.dagstuhl.de/entities/document/10.4230/LITES.8.2.5
7. (PDF) Uncertainty-aware Sensor Fusion: Integrating Bayesian Networks with Deep Perception Models for Safety-Critical Decisions - ResearchGate, accessed February 26, 2026, https://www.researchgate.net/publication/395234051_Uncertainty-aware_Sensor_Fusion_Integrating_Bayesian_Networks_with_Deep_Perception_Models_for_Safety-Critical_Decisions
8. Uncertainty quantification in automated valuation models with locally weighted conformal prediction - UiO, accessed February 26, 2026, https://www.mn.uio.no/math/english/people/aca/anderdh/conformal-prediction-avms.pdf
9. [2312.06531] Uncertainty quantification in automated valuation models with spatially weighted conformal prediction - arXiv.org, accessed February 26, 2026, https://arxiv.org/abs/2312.06531
10. Testing Marginal and Conditional Coverage in Conformal Prediction for

Non-Stationary Time Series via Value-at-Risk Backtesting - GitHub, accessed February 26, 2026, https://raw.githubusercontent.com/mlresearch/v266/main/assets/retzlaff25a/retzlaff25a.pdf
11. Uncertainty Quantification for Learning-based MPC using Weighted Conformal Prediction | George J. Pappas, accessed February 26, 2026, https://www.georgejpappas.org/wp-content/uploads/2023/12/Uncertainty_Quantification_for_Learning-based_MPC_using_Weighted_Conformal_Prediction.pdf
12. Conformal Prediction: An introduction to measuring uncertainty, accessed February 26, 2026, https://www.bbvaaifactory.com/conformal-prediction-an-introduction-to-measuring-uncertainty/
13. Counting Dense Crowds with Computer Vision AI Models - CityData.AI Blog, accessed February 26, 2026, https://blog.citydata.ai/counting-dense-crowds-with-computer-vision-ai-models/
14. Indoor Building Reconstruction from Occluded Point Clouds Using Graph-Cut and Ray-Tracing - MDPI, accessed February 26, 2026, https://www.mdpi.com/2076-3417/8/9/1529
15. [2008.11901] Multi-View Fusion of Sensor Data for Improved Perception and Prediction in Autonomous Driving - arXiv, accessed February 26, 2026, https://arxiv.org/abs/2008.11901
16. MVFusion: Multi-View 3D Object Detection with Semantic-aligned Radar and Camera Fusion - ResearchGate, accessed February 26, 2026, https://www.researchgate.net/publication/372121822_MVFusion_Multi-View_3D_Object_Detection_with_Semantic-aligned_Radar_and_Camera_Fusion
17. Holistic Multi-View Building Analysis in the Wild with Projection Pooling, accessed February 26, 2026, https://cdn.aaai.org/ojs/16393/16393-13-19887-1-2-20210518.pdf
18. (PDF) An Occlusion-Free Multi-View Building Facade Feature Reconstruction, accessed February 26, 2026, https://www.researchgate.net/publication/394679763_An_Occlusion-Free_Multi-View_Building_Facade_Feature_Reconstruction
19. (PDF) Take a Look Around: Using Street View and Satellite Images to Estimate House Prices - ResearchGate, accessed February 26, 2026, https://www.researchgate.net/publication/335707828_Take_a_Look_Around_Using_Street_View_and_Satellite_Images_to_Estimate_House_Prices
20. Symmetry-aware Façade Parsing with Occlusions - Microsoft, accessed February 26, 2026, https://www.microsoft.com/en-us/research/wp-content/uploads/2019/09/Cohen-et-al-3DV-2017.pdf
21. Real estate price prediction under asymmetric loss, accessed February 26, 2026, https://www.ism.ac.jp/editsec/aism/pdf/047_3_0401.pdf
22. Asymmetric Loss Functions - Econlib, accessed February 26, 2026, https://www.econlib.org/archives/2011/03/asymmetric_loss.html
23. Implementing a Custom Asymmetric Loss Function for Regression | by Leonidas

Gorgo, accessed February 26, 2026, https://leonidasgorgo.medium.com/implementing-a-custom-asymmetric-loss-function-for-regression-d0dae2ec3fd6
24. Asymmetric Penalties Underlie Proper Loss Functions in Probabilistic Forecasting, accessed February 26, 2026, https://www.stat.berkeley.edu/~ryantibs/papers/lossasym.pdf
25. Asymmetric Penalties Underlie Proper Loss Functions in Probabilistic Forecasting - arXiv, accessed February 26, 2026, https://arxiv.org/html/2505.00937v1
26. Addressing misclassification costs in machine learning through asymmetric loss functions, accessed February 26, 2026, https://www.nist.gov/publications/addressing-misclassification-costs-machine-learning-through-asymmetric-loss-functions
27. Properties of Optimal Forecasts under Asymmetric Loss and Nonlinearity - Rady School of Management, accessed February 26, 2026, https://rady.ucsd.edu/_files/faculty-research/timmermann/asymmetric-loss.pdf
28. Databricks Feature Store 101: A complete guide (2026) - Flexera, accessed February 26, 2026, https://www.flexera.com/blog/finops/databricks-feature-store/
29. What is a Feature Store? A Complete Guide to ML Feature Engineering | Databricks Blog, accessed February 26, 2026, https://www.databricks.com/blog/what-feature-store-complete-guide-ml-feature-engineering
30. MLOps with a Feature Store - Medium, accessed February 26, 2026, https://medium.com/data-for-ai/mlops-with-a-feature-store-8dabc845584a
31. Building a Production-Ready MLOps Monitoring Architecture: From Drift Detection to Automated Alerts | by Fareedahab | Medium, accessed February 26, 2026, https://medium.com/@fareedahab/building-a-production-ready-mlops-monitoring-architecture-from-drift-detection-to-automated-alerts-cd8d48c4c79d
32. Automating Model Drift Detection in Production ML Pipelines Using Anomaly Detection Techniques - ResearchGate, accessed February 26, 2026, https://www.researchgate.net/publication/392592315_Automating_Model_Drift_Detection_in_Production_ML_Pipelines_Using_Anomaly_Detection_Techniques
33. What Is Human In The Loop (HITL)? - IBM, accessed February 26, 2026, https://www.ibm.com/think/topics/human-in-the-loop
34. Human-in-the-Loop AI (HITL) - Complete Guide to Benefits, Best Practices & Trends for 2026, accessed February 26, 2026, https://parseur.com/blog/human-in-the-loop-ai
35. What is Human-in-the-Loop (HITL)? - DealHub.io, accessed February 26, 2026, https://dealhub.io/glossary/human-in-the-loop-hitl/?utm_sq=gcdt8nj42i
36. What Is Human-in-the-Loop Automation? A Practical Guide for Contact Centers - Balto AI, accessed February 26, 2026, https://www.balto.ai/blog/what-is-human-in-the-loop-automation/
37. Multi-View Fusion-Based Automated Full-Posture Cattle Body Size Measurement - MDPI, accessed February 26, 2026, https://www.mdpi.com/2076-2615/14/22/3190

38. Different costs for underestimation and overestimation - Stack Overflow, accessed February 26, 2026, https://stackoverflow.com/questions/44791431/different-costs-for-underestimation-and-overestimation